ORACLE
University

Oracle Cloud Infrastructure Generative AI Professional

Activity Guide

S1107455GC10

Learn more from Oracle University at ***education.oracle.com***

Copyright © 2024, Oracle and/or its affiliates.

Disclaimer

This document contains proprietary information and is protected by copyright and other intellectual property laws. The document may not be modified or altered in any way. Except where your use constitutes "fair use" under copyright law, you may not use, share, download, upload, copy, print, display, perform, reproduce, publish, license, post, transmit, or distribute this document in whole or in part without the express authorization of Oracle.

The information contained in this document is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

Restricted Rights Notice

If this documentation is delivered to the United States Government or anyone using the documentation on behalf of the United States Government, the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software" or "commercial computer software documentation" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

Trademark Notice

Oracle®, Java, MySQL, and NetSuite are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

Third-Party Content, Products, and Services Disclaimer

This documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

1011182024

Table of Contents

Generative AI Labs: Create an OCI Compute Instance 5
Get Started 6
Download the Bash Script File 7
Launch Cloud Shell 8
Generate SSH Keys 9
Create a Compute Instance 11
Update the Firewall and IP Tables on the Instance 16
Start a JupyterLab Server 18
Generative AI Labs: Explore OCI Chat Models 21
Get Started 22
Create a Jupyter Notebook 24
Generative AI Labs: Explore OCI Embedding Model 29
Get Started 30
Create a Jupyter Notebook 31
Generative AI Labs: RAG using Oracle Database 23ai - Vectorization 35
Get Started 36
Create a Jupyter Notebook 39
Generative AI Labs: RAG using Oracle Database 23ai - Retrieval 45
Get Started 46
Create a Jupyter Notebook 48
Generative AI Labs: Build a Chatbot using Oracle Database 23ai and LangChain 55
Get Started 56
Ingest PDF document to Oracle 23 ai vector store 58
Implement chat using RAG and chat history 62
Purge the Compute Instance 74
Purge the Virtual Cloud Network 75

Copyright © 2024, Oracle and/or its affiliates.

Copyright © 2024, Oracle and/or its affiliates.

Generative AI Labs: Create an OCI Compute Instance

Lab 0-01 Practices

Get Started

Overview

The Oracle Cloud Infrastructure (OCI) Compute lets you provision and manage compute hosts, known as instances. You can launch instances as needed to meet your compute and application requirements. In this lab, you will create a virtual machine/compute instance.

In this lab, you will:

- Download the bash script file
- Launch Cloud Shell
- Generate SSH keys
- Create a compute instance
- Update the firewall and ip tables on the instance
- Start a JupyterLab server

Prerequisites

- You must have access to and are familiar with navigating the OCI Console.
- The Oracle University lab team has set up all the IAM policies required for you to complete this lab.
- You have received the required credentials to successfully login into the OCI console.

Assumptions

- You are aware of your tenancy name as shared by the Oracle University lab team.
- Select the region available in the tenancy allotted to you. In this lab, **Germany Central (Frankfurt)** is considered as your region.

Copyright © 2024, Oracle and/or its affiliates.

Download the Bash Script File

A bash script or .sh file is a text file containing a series of commands that are executed in sequence by the bash (Bourne Again SHell) command-line interpreter. It is used to automate repetitive tasks or execute multiple commands in Linux or Unix-like operating systems.

In this practice, you will download and modify this script.

Tasks

1. Download the **init-genailabs.sh** from the GitHub link below or from the Guides section of the OCI Generative AI Professional course's mylearn.oracle.com page to your local machine (computer).

   https://github.com/ou-developers/ou-generativeai-pro/tree/main/labs

   Note:

   - Make sure you don't modify the code by mistake while reviewing this file. If in doubt, please download the **.sh** file again.
   - Angle brackets are used to enclose **placeholders** or **variables** in this documentation or elsewhere in the code files.

     Example: <filename> indicates that the user should replace the word this with an actual file name.
   - This bash script will be used when creating an OCI instance to install various libraries or to run necessary Linux commands.
   - The code content of the bash script is out of the scope for this lab so we will skip the related explanation. Those who are interested can research on their own.

Copyright © 2024, Oracle and/or its affiliates.

Launch Cloud Shell

The OCI Cloud Shell is a web browser–based terminal accessible from the OCI Console. It provides access to a Linux shell, with a pre-authenticated OCI CLI.

In this practice, you will access Cloud Shell via the OCI Console.

Tasks

1. Sign in to your Oracle Cloud Infrastructure Console.
2. In the Console ribbon at the top of the screen, click the Region icon to expand the menu. Ensure that you are in the correct region, **Germany Central (Frankfurt)**.
3. Click the **Cloud Shell** icon next to the Region in the Console ribbon.

   Note:

   - The OCI CLI running in the Cloud Shell will execute commands against the region selected in the Console's region selection menu when the Cloud Shell is started.
   - This displays the Cloud Shell in a "drawer" at the bottom of the console.
4. You can use the icons in the top-right corner of the Cloud Shell window to minimize, maximize, and close your Cloud Shell session.

Copyright © 2024, Oracle and/or its affiliates.

Generate SSH Keys

In this practice, you will generate SSH keys using Cloud Shell.

Tasks

1. From the OCI Console, click the **Cloud Shell** icon next to the region in the Console ribbon.

2. After the Cloud Shell has started, run the following commands:

   ```
   $ mkdir .ssh
   ```

   Note:

   - In case you get an error that says, “cannot create director: File exists”, you can skip running the above command.

   ```
   $ cd .ssh
   ```

   ```
   $ ssh-keygen -b 2048 -t rsa -f <sshkeyname>
   ```

   Replace *<sshkeyname>* above with **genailabskey**. This will be the key name you will use to connect to the compute instance you create. After entering the ssh-keygen command above, press **Enter** twice for no passphrase.

   Note:

   - If you receive an error message for the above command, enter the command manually.
   - Do not include the angle brackets <> and $ symbol when pasting code into the Cloud Shell.

3. Examine the two files that you just created by running the following command:

   ```
   $ ls -l
   ```

   Note:

   - In the output, there are two files, a private key: *<sshkeyname>* and a public key: *<sshkeyname>*.pub. In our case, it should be genailabskey and genailabskey.pub.
   - Keep the private key safe and don’t share its content with anyone. The public key will be needed for various activities and can be uploaded to certain systems as well as copied and pasted to facilitate secure communications in the cloud.

Copyright © 2024, Oracle and/or its affiliates.

4. To list the contents of the public key, run the following command:

   $ `cat <sshkeyname>.pub`

   Replace <*sshkeyname*> with **genailabskey**

   Note:

   - The angle brackets <> should not appear in your command while typing.
   - The output will look like below

     ssh-rsa
     AAAAB3NzaC1yc2EAAAABIwAAAQEArV5h5hsV7GyZL3zt6PZb5xwWqTkKj+y2SDhsuf
     S9P8k+gQxG9w4ZyV2M8vJZm4wzVuyYZOT54DL0l6y9wz0bduAB7R1CkNR1A6QaBs
     aZhZk8iDltFbD8Bzop3XwLV7XEDL5e3gMfEd4NVqv+J/V4VJfwYoihBrO9j0RGRuHE+T
     RFYndzrbSJ8PjUDFkmwDxRYdnl0idEMd7Lt4rZMjKpXCMuC5xk1rzVhl23dq3lRr3Ax9+
     K+rhWuGkRc3mQKpM5swwy9UXvl2XM7kXOBmZPfQNR8MXY3HNB3cDso23Zbwq7
     user@your-machine

5. Copy the contents of the public key (entire output) as you will require this in a subsequent step. Make sure that you remove any hard returns that may have been added when copying. The .pub key should be one line.

Copyright © 2024, Oracle and/or its affiliates.

Create a Compute Instance

In this practice, you will launch a compute instance and connect to it.

Tasks

1. From the OCI Console **Main Menu**, under **Compute**, click **Instances**.

2. Click **Create instance**.

3. In the **Create compute instance** dialog box, populate the following information:

   a. **Name**: GEN-AI-LABS

   b. **Create in compartment**: Select <*your compartment*>

   c. **Placement (Availability domain)**: AD 1

      Click **Show advanced options** and select **On-demand capacity** under Capacity type.

   d. **Image & shape:**

      Image: Oracle Linux 8 Image build: 2024.09.30-0

      Shape: Select VM.Standard.E5.Flex (Under AMD), **Number of OCPUs**: 2, **Amount of memory (GB):** 24

   e. **Primary VNIC information:**

      Primary network: Create new virtual cloud network.

      New virtual cloud network name: GENAILABS-VCN

      Subnet: Create new public subnet.

      New subnet name: GENAILABS-SNET

   f. **Add SSH keys**: Paste public keys.

      SSH Keys: <*public key*>

      (Paste the public key which you copied in Step 5 of Generate SSH Keys practice.)

   g. **Boot volume**: Specify a custom boot volume size

Copyright © 2024, Oracle and/or its affiliates.

Boot volume size (GB) : 100 GB (Default is 50 GB)

h. **Show advanced option**: Choose cloud-init script file

Browse and upload the file **init-genailabs.sh** which you saved in Step 1 of Download the Bash Script File practice.

4. Click **Create**.

   You will see that the Instance is created successfully, and the state is **Running**.

5. Copy the Public IP corresponding to the **GEN-AI-LABS** instance and paste it in the Notepad.

6. Click on the **Instance name** and then under **Primary VNIC** (to the right side) on the **Instance information** section, click on the **Subnet** GENAILABS-SNET. This will take you to **Subnet Details** page.

7. Under **Security Lists,** click on the **Default Security List for GENAILABS-VCN.**

   Here, you need to open port **8888, 8501 and 1521**.

8. Click **Add Ingress Rules**.

   Note:

   - Adding the Ingress rules will allow incoming traffic to reach a specific port from any IP address. This makes a service, such as Jupyter Notebook, accessible from the internet.
   - Users can access the Jupyter Notebook interface running on the port 8888, Streamlit on the port 8501 and Oracle Database on the port 1521.

Copyright © 2024, Oracle and/or its affiliates.

9. In the **Add Ingress Rules** dialog box, populate the following information:

   a. **Source Type**: CIDR

   b. **Source CIDR**: Enter the public IP address of your local machine where the Jupyter server will be opened from a browser (for example, if the public IP address is 123.123.456.456 then enter 123.123.456.456/32)

   c. **IP Protocol**: TCP

   d. **Destination Port Range**: 8888,8501,1521

   Note:

   - The source CIDR is the IP of your local machine. This IP is dynamically assigned and can be fetched from searching what's is my ip in google search or from **ip4.me** URL.
   - The **/32** means only this specific IP address is allowed, as it represents a subnet mask with all 32 bits fixed (a single IP).
   - Do not select the **Stateless** check box. The **Source Port Range** field is set to **All** by default.
   - Do not leave any space after the comma(,) while specifying the ports 8888,8501,1521. It will show Invalid input otherwise.
   - Above setup is not recommended for production environments unless additional security measures, such as firewall rules, VPNs, or access control lists (ACLs), are in place.

10. Click **Add Ingress Rule**.

    You can see that three new route rules are successfully added.

11. Click the **Cloud Shell** icon next to the Region at the top of the screen.

12. Run the following command using SSH to connect to your instance:

```
$ ssh -i <private_key_file> <username>@<public-ip-address>
```

    a. The <*private_key_file*> is the full path and name of the file that contains the private key associated with the instance you want to access.

    b. The <*username*> is the default user opc.

    c. The <*public-ip-address*> is the public IP address of the instance.

Copyright © 2024, Oracle and/or its affiliates.

Note:

- You had copied Public IP in the step 5 earlier.
- Enter **yes** in response to - Are you sure you want to continue connecting (yes/no)?

You are now connected to the instance **GEN-AI-LABS**.

Troubleshooting

- The bash script is running in the background and if you want to see what's going on in the bash script as it runs you can use the following command in the Cloud Shell

  ```
  $ sudo tail -f /var/log/cloud-init-output.log
  ```

  This command is used to **continuously monitor the last part of the cloud-init log file** in real-time. Once the entire script is done executing you should see something similar to the below

  ```
  ===== Cloud-Init Script Completed Successfully =====

  ===== Cloud-Init Script Completed Successfully =====

  Cloud-init v. 23.4-7.0.1.el8_10.7 running 'modules:final' at
  Tue, 22 Oct 2024 03:48:44 +0000. Up 18.79 seconds.

  Cloud-init v. 23.4-7.0.1.el8_10.7 finished at Tue, 22 Oct 2024
  04:05:46 +0000. Datasource DataSourceOracle.  Up 1233.81 seconds
  ```

  Press Ctrl + C to terminate or interrupt.

  The bash script should take around 20 mins or so, which can differ in real-time. Unless you are observing the script results and see the completion message, you can come back and recheck with the above command for the completion message after 20 mins.

- Alternatively, if you missed above and if you want to view the entire content of the cloud-init log file in a scrollable interface, you can use the command below in the Cloud Shell

  ```
  $ less /var/log/cloud-init-output.log
  ```

  Once inside less, you can use Up/Down arrows or your mouse to navigate.

  Press q to quit and return to the terminal.

Copyright © 2024, Oracle and/or its affiliates.

Note:

- It's advised to run this command (less) only after the previous command (tail) to confirm the script is done executing, so you see the final log and not something intermediate.
- These commands are important if the script fails, and you want to check what went wrong.
- In case, you see any database related issues during the script execution such as ORA-01155 or so, please purge the compute instance and once it's terminated, recreate it using the script after some time. Purge instructions are in last lab – Lab5 guide.
- Also, if you are not able to run the Jupyter after the script execution, check the cloud-init log file and see if any step was not done successfully. Again, please purge the compute instance and once it's terminated, recreate it using the script after some time. Purge instructions are in last lab – Lab5 guide.

Copyright © 2024, Oracle and/or its affiliates.

Update the Firewall and IP Tables on the Instance

Firewalld and iptables are tools used for managing firewall rules and controlling network traffic in Linux systems. They serve similar purposes but work differently and are commonly found in various Linux distributions.

In this practice, you will configure the firewall and update the iptables.

Tasks

1. On the OCI Console, click the **Cloud Shell** icon at the top of the screen.

2. While connected to your compute instance via SSH, run the following commands:

   a. To create firewall rules to allow access to the necessary ports:

   ```
   $ sudo firewall-cmd --zone=public --add-port=8888/tcp --
      permanent
   ```

   ```
   $ sudo firewall-cmd --zone=public --add-port=8501/tcp --
      permanent
   ```

   ```
   $ sudo firewall-cmd --zone=public --add-port=1521/tcp --
      permanent
   ```

   b. To apply any changes made to the firewall configuration:

   ```
   $ sudo firewall-cmd --reload
   ```

   c. To allow incoming TCP traffic on the ports through the iptables firewall on a Linux system:

   ```
   $ sudo iptables -A INPUT -p tcp --dport 8888 -j ACCEPT
   ```

   ```
   $ sudo iptables -A INPUT -p tcp --dport 8501 -j ACCEPT
   ```

   ```
   $ sudo iptables -A INPUT -p tcp --dport 1521 -j ACCEPT
   ```

   d. To switch to the labs directory:

   ```
   $ cd labs/
   ```

   e. To reload or apply the changes made in the .bashrc file during the bash script execution in the current shell session:

   ```
   $ source $HOME/.bashrc
   ```

Copyright © 2024, Oracle and/or its affiliates.

Note:

- The .bashrc is a configuration file that resides in the home directory of a user and is used to customize the user's bash environment, such as setting environment variables, creating aliases, and defining functions.

f. To validate that Python 3.11.9 is installed:

```
$ python --version
```

g. To validate if the files were downloaded. You can run following command:

```
$ tree $HOME/labs
```

The output should like following:

```
[opc@genai-final labs]$ tree $HOME/labs
/home/opc/labs
├── config.txt
├── init-genailabs.sh
├── install.sh
├── LoadProperties.py
├── pdf-docs
│   └── oci-ai-foundations.pdf
└── txt-docs
    └── faq.txt


2 directories, 6 files
```

Copyright © 2024, Oracle and/or its affiliates.

Start a JupyterLab Server

JupyterLab is an interactive development environment for running Jupyter notebooks.

In this practice, you will start a JupyterLab server and change the config.txt file.

Tasks

1. On the OCI Console, click the **Cloud Shell** icon at the top of the screen.

2. While connected to your compute instance via SSH, run the following command to start a JupyterLab server in a way that makes it accessible remotely, and it runs the server in the background.

   ```
   $ nohup jupyter-lab --no-browser --ip 0.0.0.0 --
   NotebookApp.token='' --NotebookApp.password='' --port 8888 &
   ```

   Note:

   - nohup allows the Jupyter Lab process to continue running even if you log out or close the terminal. Output from the command is typically written to a file called nohup.out unless you specify a different file for output.
   - --no-browser prevents Jupyter Lab from attempting to open a web browser when it starts. This is useful when running Jupyter Lab on a remote server, as you typically access it through a browser on your local machine.
   - Token-based authentication and password protection has been disabled by setting the token to an empty string ('').
   - & runs the entire command in the background, allowing you to continue using the terminal for other tasks.

3. Now open a browser on your computer and type the following

   `http://<public-ip-address>:8888`

   (Replace the *<public-ip-address>* with **GEN-AI-LABS** instance's public ip with what you copied in the Step 5 of Create a Compute Instance practice.)

   Note:

   - You should now see JupyterLab screen along with some files and folder.
   - In case you get a secure connection message kindly ignore the message " <IP address> does not support a secure connection. Click continue to site.

Copyright © 2024, Oracle and/or its affiliates.

4. Open the **config.txt** file and replace the compartment_ocid with your compartment_ocid and save the config.txt file. Don't forget the quotes.

   "compartment_ocid":"<ocid1.compartment.oc1..xxxxxxxxx……>"

   Note:

   - To find your compartment_ocid, from the OCI Console **Main Menu**, under **Identity & Security**, click **Compartments** and then copy **OCID**.

Troubleshooting

- Use the following steps, if the JupyterLab fails to start. While connected to your compute instance via SSH through the **Cloud Shell**, and once in the labs folder, check for the running processes using

  ```
  $ ps -a
  ```

  ```
   PID TTY          TIME CMD
   805242 pts/1    00:00:00 ps
  ```

  If you see Jupyter running, note the Process ID or PID. Alternatively, you can also check whether the port is being used through the command below

  ```
  $ sudo lsof -i :8888
  ```

  ```
  COMMAND     PID USER   FD   TYPE  DEVICE SIZE/OFF NODE NAME
  jupyter-l 74664  opc    6u  IPv4  184826      0t0  TCP *:ddi-
  tcp-1 (LISTEN)
  jupyter-l 74664  opc   49u  IPv4 6446727      0t0  TCP genai-
  comb6.subnet09191445.vcn09191445.oraclevcn.com:ddi-tcp-1->104-
  202-165-84.dyn.grandenetworks.net:54213 (ESTABLISHED)
  ```

  You might see multiple lines in the output, but the PID should be the same, again make a note of the PID.

- Kill the process by using the command below

  ```
  $ kill -9 <PID>
  ```

  Example:

  ```
  $ kill -9 74664
  ```

  Above should kill the process and if you run the earlier two commands, you shouldn't see any PID associated with Jupyter anymore.

Copyright © 2024, Oracle and/or its affiliates.

- Check for the ports which the firewall allows

  ```
  $ sudo firewall-cmd --zone=public --list-ports
  ```

  Above should show 1521/tcp 8501/tcp 8888/tcp.

  If not, redo the step 2-a and 2-b in the tasks of Update the Firewall and IP Tables on the Instance practice.
- You can now run the Jupyter again using the nohup command as shown in the Step 2 in tasks of Start a JupyterLab Server practice.

Copyright © 2024, Oracle and/or its affiliates.

Generative AI Labs: Explore OCI Chat Models

Lab 1-01 Practices

Get Started

Overview

The Oracle Cloud Infrastructure (OCI) Generative AI is a fully managed Oracle Cloud Infrastructure service that provides a set of state-of-the-art, customizable large language models (LLMs) that cover a wide range of use cases, including chat. Through OCI Generative AI Chat model you can ask questions and get conversational responses through an AI chatbot.

Model: An AI model trained on large amounts of data which takes inputs that it hasn't seen before and generates new content.

Chat Models: The following pretrained foundational models are available in OCI Generative AI for chat:

- cohere.command-r-16k
- cohere.command-r-plus
- meta.llama-3.1-70b-instruct
- meta.llama-3.1-405b-instruct

The cohere.command-r-plus (Command R+) and cohere.command-r-16k (Command R) chat models both belong to the Cohere Command R family of LLMs.

The meta.llama-3.1-405b-instruct and meta.llama-3.1-70b-instruct belong to the Meta Llama 3.1 family of LLMs.

Prompt/Message: Strings of text in natural language used to instruct or extract information from a large language model.

Token: A token is a word, part of a word, or a punctuation. For example, apple is one token, and friendship is two tokens (friend and ship). When uses as model parameter, it specifies the maximum number of tokens (words and subwords) that the AI model can generate in response to the prompt.

Temperature: The level of randomness used to generate the output text. To generate a similar output for a prompt every time that you run that prompt, use 0. To generate a random new text for that prompt, increase the temperature.

Frequence Penalty: A penalty that is assigned to a token when that token appears frequently. High penalties encourage fewer repeated tokens and produce a more random output.

Top p: A sampling method that controls the cumulative probability of the top tokens to consider for the next token. Assign p a decimal number between 0 and 1 for the probability.

Copyright © 2024, Oracle and/or its affiliates.

For example, enter 0.75 for the top 75 percent of probability mass to be considered by the model. Set p to 1 to consider all tokens.

Top k: A sampling method in which the model chooses the next token randomly from the top k most likely tokens. A higher value for k generates more random output, which makes the output text sound more natural.

In this lab, you will:

- Create a Jupyter notebook and run the chat model code.

Prerequisites

- You have completed all the practices from **Lab0-01**.

Assumptions

- You are familiar with Python coding language.
- You are on the JupyterLab screen in your browser and have already updated the **config.txt** as directed in the step 4 of the **Start a JupyterLab Server** practice in the **Lab0-01**.
- You can see the **LoadProperties.py, config.txt, faq.txt (inside txt-docs folder)** and **oci-ai-foundations.pdf (inside pdf-docs folder)** with few other files in the File Browser section on the left-hand side.

Copyright © 2024, Oracle and/or its affiliates.

Create a Jupyter Notebook

JupyterLab is the latest web-based interactive development environment for notebooks, code, and data. Its flexible interface allows users to configure and arrange workflows in data science and machine learning.

Notebook provides a simple, linear notebook interface where you can write and execute code cells, see output inline, and use markdown for text.

config.txt is a configuration file used to specify variable values in a Json format. You will see it as below

```
{"model_name":"cohere.command-r-16k",
"embedding_model_name":"cohere.embed-english-v3.0",
"endpoint":"https://inference.generativeai.eu-frankfurt-
1.oci.oraclecloud.com",
"compartment_ocid":"ocid1.compartment.oc1…"
}
```

LoadProperties.py file defines a Python class named `LoadProperties` that reads configuration settings from a config.txt file and provides methods to access those settings.

```
class LoadProperties:

    def __init__(self):

        import json
        # reading the data from the file
        with open('config.txt') as f:
            data = f.read()

        js = json.loads(data)

        self.model_name = js["model_name"]
        self.endpoint = js["endpoint"]
        self.compartment_ocid = js["compartment_ocid"]
        self.embedding_model_name=js["embedding_model_name"]

    def getModelName(self):
            return self.model_name

    def getEndpoint(self):
            return self.endpoint
```

Copyright © 2024, Oracle and/or its affiliates.

```
    def getCompartment(self):
            return self.compartment_ocid

    def getEmbeddingModelName(self):
            return self.embedding_model_name
```

Note:

- `js = json.loads(data)` converts the JSON string into a Python dictionary named `js`.
- `self.model_name, self.endpoint, self.compartment_ocid`, and `self.embedding_model_name` are set using the parsed values from the JSON object.
- Getter methods such as `getModelName, getEndpoint, getCompartment, getEmbeddingModelName` provide access to the values stored in the class attributes.

In this practice, you will create a notebook file. You will then write and execute code to implement a OCI Generative AI Chat model demonstration. You will also utilize the config.txt and LoadProperties.py to successfully write and execute the code.

Tasks

1. In the Console ribbon at the top of the JupyterLab Server screen in your browser, click on the **File** > **New** > **Notebook**.
2. This will open a **Select Kernel** dialog box. Leave the default kernel (**Python 3**) as it is and click on **Select**.
3. You will see an **Untitled.ipynb** notebook created. Right click on it and select **Rename**. Rename the notebook to **Lab1.ipynb**.
4. Once renamed, in the cell start copy these code snippets one by one and execute them by pressing **shift + enter**. Once executed you will see that the cell has been assigned a number. You may see an output below the cell depending on the code written.

   a. Cell 1

```
# Import the necessary libraries
import oci
from LoadProperties import LoadProperties

# Setup basic variables
properties = LoadProperties()
```

Copyright © 2024, Oracle and/or its affiliates.

`LoadProperties()` class reads the configuration values of `model_name, endpoint, compartment_ocid, embedding_model_name` from the config.txt file.

b. Cell 2

```
# Use Instance Principals for Authentication
signer =
oci.auth.signers.InstancePrincipalsSecurityTokenSigner()
```

Using instance principal authentication, you can authorize an instance to make API calls on Oracle Cloud Infrastructure services. It removes the need to configure user credentials or a configuration file.

c. Cell 3

```
# Initialize the Generative AI Client
generative_ai_inference_client =
oci.generative_ai_inference.GenerativeAiInferenceClient(
    config={},
    signer=signer,
    service_endpoint=properties.getEndpoint(),
    retry_strategy=oci.retry.NoneRetryStrategy(),
    timeout=(10, 240)
)
```

It initializes the OCI Generative AI client using OCI SDK to authenticate and establish the connection. The client will be used to make requests to the OCI Generative AI service.

`config={}` is used to specify the configuration for connecting to OCI. An empty dictionary `{}` is provided because authentication is handled using signer in the previous step, making it unnecessary to provide a detailed configuration file.

`properties.getEndpoint()` retrieves this URL from a configuration file (config.txt), using a method defined in the `LoadProperties` class. The model is hosted at this URL.

`oci.retry.NoneRetryStrategy()` means that if a request fails, the client will not automatically retry it.

`timeout=(10, 240)` specifies the timeouts for the client. 10 seconds is the connection timeout, meaning how long the client waits for a connection to be

Copyright © 2024, Oracle and/or its affiliates.

established with the server. 240 seconds is the read timeout, meaning how long the client waits for a response after a connection is established.

d. Cell 4

```
# Set Up Chat Request Details
chat_detail = oci.generative_ai_inference.models.ChatDetails()
chat_request =
oci.generative_ai_inference.models.CohereChatRequest()
chat_request.message = "How does a telescope work"
chat_request.max_tokens = 600
chat_request.temperature = 1
chat_request.frequency_penalty = 0
chat_request.top_p = 0.75
chat_request.top_k = 0
```

We set up a chat detail object and a chat request object for passing the user query and other model parameters.

`chat_detail` instance of `oci.generative_ai_inference.models.ChatDetails()` that will encapsulate the complete details of the chat request.

`chat_request` instance of `oci.generative_ai_inference.models.CohereChatRequest()`, which is used to configure the specifics of the message or prompt you want to send to the chat model.

e. Cell 5

```
# Configure Chat Details
chat_detail.serving_mode =
oci.generative_ai_inference.models.OnDemandServingMode(model_id=
properties.getModelName())
chat_detail.chat_request = chat_request
chat_detail.compartment_id = properties.getCompartment()
```

We then configure the chat details such as serving mode (OnDemand) with the model id, previously defined chat request and the OCI compartment id for the chat operation.

`OnDemandServingMode` allows the model to be used on demand, meaning the model is accessed as needed without pre-deployment.

Copyright © 2024, Oracle and/or its affiliates.

`chat_detail.chat_request = chat_request` assigns the chat request (configured earlier) to the `chat_detail` object.

f. Cell 6

```
# Make the Chat Request and Retrieve the Response
chat_response = generative_ai_inference_client.chat(chat_detail)

# Print result
print("*****************ChatResult*****************")
print(vars(chat_response))
```

We call the chat method of the inference client and pass the chat detail object which we created earlier. We finally print the response from the chat model and display the response content in a readable format using vars, which converts the object attributes into a dictionary.

`chat_response = generative_ai_inference_client.chat(chat_detail)` method sends the `chat_detail` object to the OCI Generative AI service.

`chat()` method then processes this information and communicates with the AI model, which generates a response based on the provided input.

`vars()` to convert the `chat_response` object into a dictionary-like format.

Note:

- The chat response object has status code: 200 if the request is successful.
- Please check the “text” field in the response object payload to observe the response to your query.

5. Once all the cells above run successfully, go to **File > Save Notebook** to save the notebook.

Copyright © 2024, Oracle and/or its affiliates.

Generative AI Labs: Explore OCI Embedding Model

Lab 2-01 Practices

Get Started

Overview

The OCI Generative AI embedding models transforms each phrase, sentence, or paragraph that you input, into an array with 384 (light models) or 1024 numbers, depending on the embedding model that you select.

You can use these embeddings for finding similarity in phrases that are similar in context or category. Embeddings are typically stored in a vector database. Embeddings are mostly used for semantic searches where the search function focuses on the meaning of the text that it's searching through rather than finding results based on keywords.

Embedding Models: The following pretrained models are available for creating text embeddings:

- cohere.embed-english-v3.0
- cohere.embed-multilingual-v3.0
- cohere.embed-english-light-v3.0
- cohere.embed-multilingual-light-v3.0

Truncate: This is an embedding model parameter which defines whether to truncate the start or end tokens in a sentence, when that sentence exceeds the maximum number of allowed tokens. For example, a sentence has 516 tokens, but the maximum token size is 512. If you select to truncate the end, the last 4 tokens of that sentence are cut off.

In this lab, you will:

- Create a Jupyter notebook and run the embedding model code.

Prerequisites

- You have completed all the practices from **Lab0-01** and **Lab1-01**.

Assumptions

- You are familiar with Python coding language.
- You are on the JupyterLab screen in your browser and have already updated the **config.txt** as directed in the step 4 of the **Start a JupyterLab Server** practice in the **Lab0-01**.
- You can see the **LoadProperties.py, config.txt, faq.txt (inside txt-docs folder)** and **oci-ai-foundations.pdf (inside pdf-docs folder)** with few other files in the File Browser section on the left-hand side.

Copyright © 2024, Oracle and/or its affiliates.

Create a Jupyter Notebook

JupyterLab is the latest web-based interactive development environment for notebooks, code, and data. Its flexible interface allows users to configure and arrange workflows in data science and machine learning.

Notebook provides a simple, linear notebook interface where you can write and execute code cells, see output inline, and use markdown for text.

Tasks

1. In the Console ribbon at the top of the JupyterLab Server screen in your browser, click on the **File** > **New** > **Notebook**.
2. This will open a **Select Kernel** dialog box. Leave the default kernel (**Python 3**) as it is and click on **Select**.
3. You will see an **Untitled.ipynb** notebook created. Right click on it and select **Rename**. Rename the notebook to **Lab2.ipynb**.
4. Once renamed, in the cell start copy these code snippets one by one and execute them by pressing **shift + enter**. Once executed you will see that the cell has been assigned a number. You may see an output below the cell depending on the code written.

   a. Cell 1

   ```
   # Import the necessary libraries
   import oci
   from LoadProperties import LoadProperties

   # Setup basic variables
   properties = LoadProperties()
   ```

   `LoadProperties()` class reads the configuration values of `model_name`, `endpoint`, `compartment_ocid`, `embedding_model_name` from the config.txt file.

   b. Cell 2

   ```
   # Use Instance Principals for Authentication
   signer =
   oci.auth.signers.InstancePrincipalsSecurityTokenSigner()
   ```

Copyright © 2024, Oracle and/or its affiliates.

Using instance principal authentication, you can authorize an instance to make API calls on Oracle Cloud Infrastructure services. It removes the need to configure user credentials or a configuration file.

c. Cell 3

```
# Initialize the Generative AI Client
generative_ai_inference_client =
oci.generative_ai_inference.GenerativeAiInferenceClient(
    config={},
    signer=signer,
    service_endpoint=properties.getEndpoint(),
    retry_strategy=oci.retry.NoneRetryStrategy(),
    timeout=(10, 240)
)
```

It initializes the OCI Generative AI client using OCI SDK to authenticate and establish the connection. The client will be used to make requests to the OCI Generative AI service.

`config={}` is used to specify the configuration for connecting to OCI. An empty dictionary `{}` is provided because authentication is handled using signer in the previous step, making it unnecessary to provide a detailed configuration file.

`properties.getEndpoint()` retrieves this URL from a configuration file (config.txt), using a method defined in the `LoadProperties` class. The model is hosted at this URL.

`oci.retry.NoneRetryStrategy()` means that if a request fails, the client will not automatically retry it.

`timeout=(10, 240)` specifies the timeouts for the client. 10 seconds is the connection timeout, meaning how long the client waits for a connection to be established with the server. 240 seconds is the read timeout, meaning how long the client waits for a response after a connection is established.

d. Cell 4

```
# Set up Embedding Request
# inputs = ["What is the capital of France?"]
inputs = ["France"]
embed_text_detail =
oci.generative_ai_inference.models.EmbedTextDetails()
```

Copyright © 2024, Oracle and/or its affiliates.

```
embed_text_detail.serving_mode =
oci.generative_ai_inference.models.OnDemandServingMode(
    model_id=properties.getEmbeddingModelName()
)
embed_text_detail.inputs = inputs
embed_text_detail.truncate = "NONE"
embed_text_detail.compartment_id = properties.getCompartment()
```

We set up a chat detail object and a chat request object for passing the user query and other model parameters.

`embed_text_detail` is an instance of instance of `EmbedTextDetails` which holds all the necessary details for the embedding request.

`embed_text_detail.inputs` assigns the text inputs for which embeddings need to be generated.

`embed_text_detail.truncate` specifies how text inputs should be truncated if they exceed the model's maximum token length. `"NONE"` means that no truncation should occur, and the input is expected to fit within the model's maximum token limit.

e. Cell 5

```
# Make the Embedding Request and Retrieve the Response
embed_text_response =
generative_ai_inference_client.embed_text(embed_text_detail)
# Print result
print("*****************Embed Texts Result*****************")
print(embed_text_response.data)
```

`embed_text()` method sends the `embed_text_detail` object to the OCI Generative AI service, which processes the input text and returns a vector representation (embedding).

`embed_text_response` stores the response from the AI model, which contains the generated embeddings.

Note:

- You will see the list of vectors that represent the input text. Each number in the vector corresponds to a feature that the model has identified for "France."

5. Once all the cells above run successfully, go to **File > Save Notebook** to save the notebook.

Copyright © 2024, Oracle and/or its affiliates.

Copyright © 2024, Oracle and/or its affiliates.

Generative AI Labs: RAG using Oracle Database 23ai - Vectorization

Lab 3-01 Practices

Get Started

Overview

This lab guides you through the code required to load and parse a text FAQ from a file named **faq.txt** from a local folder, split them into segments, and then create embeddings and ingest them into the vector database. You can use these embeddings for finding similarity in phrases that are similar in context or category. Embeddings are typically stored in a vector database which in this case **Oracle Database 23ai**.

LLM: Large Language Model (LLM) is a probabilistic model of text. A probabilistic model is a type of mathematical model that represents uncertain events or data points by assigning probabilities to different possible outcomes. It doesn't make exact predictions but rather gives a range of possible outcomes and the likelihood (probability) of each of the outcomes.

In simpler terms, it's like saying, "What word is likely to come next?" and making a prediction based on patterns and rules the model has learned from a large body of text.

Vector: A vector in the context of LLMs is an array of numbers that represents textual data in a way that the model can understand and manipulate. Vectors are often multi-dimensional arrays (lists of floats) that capture the semantic meaning of words, phrases, sentences, or entire documents.

For example, a word like "cat" might be represented by a 768-dimensional vector like [0.012, -0.238, 0.876, ..., -0.345]. Each number in this vector corresponds to a feature that the model has learned from the training data.

Similarity search: It is a technique used to find items that are semantically or conceptually similar to a given input item, often by comparing vector representations of the items. It is widely used in applications like information retrieval, recommendation systems, image recognition, and natural language processing (NLP).

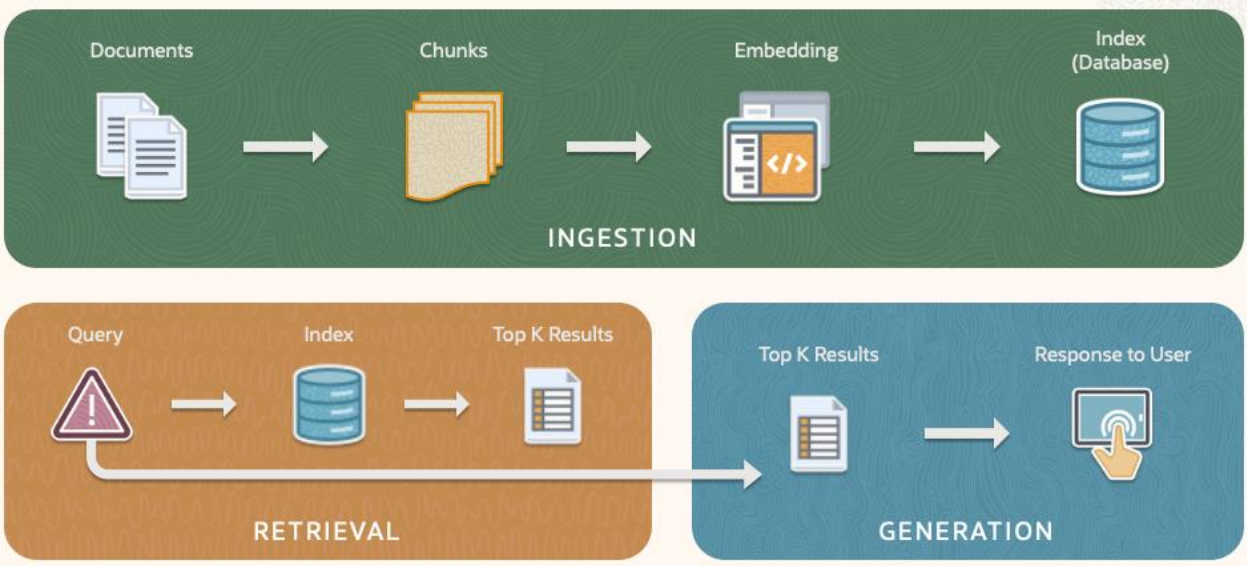
To measure how close or similar two vectors are in this space, **similarity metrics** are used. Some common metrics include Cosine Similarity, Euclidean Distance, Dot Product and Manhattan Distance (L1 distance). The process of similarity search includes:

- Convert each item (e.g., documents, images, or products) in the dataset into vectors using a model (e.g., language models for text, neural networks for images).
- Convert the query (user input) into a vector using the same model.
- Compare the query vector to each vector in the dataset using a similarity metric.

  Return the items whose vectors are most similar to the query vector.

Copyright © 2024, Oracle and/or its affiliates.

RAG: Retrieval Augmented Generation (RAG) is a technique that enhances LLMs by integrating Similarity Search. This enables use cases such as a corporate chatbot responding with private company knowledge to make sure it's giving answers that are up-to-date and tailored to your business.



Let's see the above process step by step

Ingestion: This is the first phase where documents are ingested into the system.

- Documents: The original data corpus, which can consist of a vast collection of text documents
- Chunks: The documents are broken down into smaller, more manageable pieces, often referred to as chunks. This is typically done to improve the efficiency of processing and to focus on relevant sections of the text.
- Embeddings: Each chunk is then transformed into a mathematical representation called an embedding. These embeddings capture the semantic information of the text and allow for comparisons to be made in a numerical space.
- Index: The embeddings are indexed in a database that facilitates quick retrieval. The index is a data structure that allows the system to find and retrieve embeddings efficiently when a query is made.

Retrieval: In this phase, the system uses the indexed data to find relevant information.

- Query: The user's input or question that needs to be answered
- Index: The system uses the query to search through the indexed embeddings from the ingestion phase to find the most relevant chunks.

Copyright © 2024, Oracle and/or its affiliates.

- Top K: From the retrieval process, the system selects the top K (a preset number) most relevant results. These are the chunks that are most likely to contain information relevant to the query.

Generation: This is the final phase where the system generates a response based on the information retrieved.

- Top K: The selected chunks from the retrieval phase are fed into the generative model.
- Response: The generative model, often a neural network like a Transformer, uses the context provided by the Top K chunks to generate a coherent and contextually relevant response to the query.

sentence-transformers: `sentence-transformers` is a library that provides an easy-to-use interface for generating embeddings from text using various pre-trained models. The library is built on top of Hugging Face Transformers and provides models specifically tuned for semantic similarity and sentence embeddings. It loads the **all-MiniLM-L12-v2** encoder model and uses it internally to transform sentences into vectors.

In this lab, you will:

- Create a Jupyter notebook to split a folder of text FAQs into chunks and store them as vectors using an encoder in a vector database (Oracle Database 23ai).

Prerequisites

- You have completed all the practices from **Lab0-01**.

Assumptions

- You are familiar with Python coding language and SQL queries.
- You are on the JupyterLab screen in your browser and have already updated the **config.txt** as directed in the step 4 of the **Start a JupyterLab Server** practice in the **Lab0-01**.
- You can see the **LoadProperties.py, config.txt, faq.txt (inside txt-docs folder)** and **oci-ai-foundations.pdf (inside pdf-docs folder)** with few other files in the File Browser section on the left-hand side.

Copyright © 2024, Oracle and/or its affiliates.

Create a Jupyter Notebook

JupyterLab is the latest web-based interactive development environment for notebooks, code, and data. Its flexible interface allows users to configure and arrange workflows in data science and machine learning.

Notebook provides a simple, linear notebook interface where you can write and execute code cells, see output inline, and use markdown for text.

Tasks

1. In the Console ribbon at the top of the JupyterLab Server screen in your browser, click on the **File** > **New** > **Notebook**.

2. This will open a **Select Kernel** dialog box. Leave the default kernel (**Python 3**) as it is and click on **Select**.

3. You will see an **Untitled.ipynb** notebook created. Right click on it and select **Rename**. Rename the notebook to **Lab3.ipynb**.

4. Once renamed, in the cell start copy these code snippets one by one and execute them by pressing **shift + enter**. Once executed you will see that the cell has been assigned a number. You may see an output below the cell depending on the code written.

   a. Cell 1

```
# Read the source file and store chunks in an array
import os

def loadFAQs(directory_path):
   faqs = {}


   for filename in os.listdir(directory_path):
      if filename.endswith(".txt"):  # assuming FAQs are in .txt
files
         file_path = os.path.join(directory_path, filename)


         with open(file_path) as f:
            raw_faq = f.read()


         filename_without_ext = os.path.splitext(filename)[0]  #
remove .txt extension
```

Copyright © 2024, Oracle and/or its affiliates.

```
        faqs[filename_without_ext] = [text.strip() for text in
raw_faq.split('=====')]

   return faqs
```

The function will open all the .txt files in a specified folder which is **txt-docs**, read them, split the content using the `========` separator. It will then put all the resulting chunks in an array.

The array is stored inside a dictionary with the file name used as the key. This will be useful later if many other FAQ files are available inside the folder, helping to differentiate between the sources.

b. Cell 2

```
# Store data from txt-docs
faqs = loadFAQs('./txt-docs')
faqs
```

We will call the `loadFAQs` function above with our data stored in txt-docs folder.

c. Cell 3

```
docs = [{'text': filename + ' | ' + section, 'path': filename}
for filename, sections in faqs.items() for section in sections]

# Sample the resulting data
docs[:2]
```

We will now prepare the source data to arrange the above dictionary in a way that is easy to ingest in the vector database. This will store the data in the following format:

```
[{'text': 'faq | Who are you and what can you do?\n\nI am DORA,
the Digital ORacle Assistant, a digital assistant working for
Oracle EMEA. I can answer questions about Oracle Cloud (OCI) and
especially about the Free Trial and Always Free programs.',

  'path': 'faq'}]
```

As you can see, for each chunk, we prepend the name of the source file to the text content, in our case, "faq |". This is a very simple way to ensure that some context is preserved with each chunk and vectorized later. It will help tremendously in the retrieval stage, when vector distances will be calculated between the question and each chunk.

Copyright © 2024, Oracle and/or its affiliates.

Also, we state the name of the source file in the path component. It will be useful later if we want to display a link back to the source when using a chatbot.

d. Cell 4

```
# Connect to the Oracle Database 23ai
un = "vector"
pw = "vector"
cs = "localhost/FREEPDB1"

import oracledb

connection = oracledb.connect(user=un, password=pw, dsn=cs)
```

`oracledb.connect()` function establishes a connection to an Oracle database using the provided credentials and connection details. The function takes username, password and dsn. The DSN (data source name) specifies the host, port and database service name to connect to. You can refer the bash script **init-genailabs.sh** which you earlier downloaded in step 1 of the tasks of Download the Bash Script File practice in **Lab0-01**.

e. Cell 5

```
table_name = 'faqs'

with connection.cursor() as cursor:
    # Create the table
    create_table_sql = f"""
        CREATE TABLE IF NOT EXISTS {table_name} (
            id NUMBER PRIMARY KEY,
            payload CLOB CHECK (payload IS JSON),
            vector VECTOR
        )"""
    try:
        cursor.execute(create_table_sql)
    except oracledb.DatabaseError as e:
        raise

    connection.autocommit = True
```

We need a table inside our database to store our vectors and metadata.

Copyright © 2024, Oracle and/or its affiliates.

f. Cell 6

```
from sentence_transformers import SentenceTransformer
encoder = SentenceTransformer('all-MiniLM-L12-v2')
```

We need an encoder to handle the vectorization for us. `all-MiniLM-L12-v2` is a specific pre-trained model that is designed to be an encoder. It is based on the MiniLM (Mini Language Model) architecture, which is a lightweight version of transformer models like BERT.

Note:

- Ignore the warning saying IProgress not found., among others.

g. Cell 7

```
import array

# Define a list to store the data
data = [
   {"id": idx, "vector_source": row['text'], "payload": row}
   for idx, row in enumerate(docs)
]

# Collect all texts for batch encoding
texts = [f"{row['vector_source']}" for row in data]

# Encode all texts in a batch
embeddings = encoder.encode(texts, batch_size=32,
show_progress_bar=True)

# Assign the embeddings back to your data structure
for row, embedding in zip(data, embeddings):
   row['vector'] = array.array("f", embedding)
```

We go through all our chunks (stored in the docs dictionary) and encode the text content. Now we have a structure with all our chunks, including its context—the source file name, in this simple example—and the vector representation for each of them.

`encoder.encode()` method is used to generate embeddings (vector representations) for the input texts. It uses the `encoder` object, which is an instance of the `SentenceTransformer` class, loaded with a specific model (e.g., all-MiniLM-L12-v2).

Copyright © 2024, Oracle and/or its affiliates.

`batch_size=32` means that 32 sentences will be processed in parallel during each batch, which can be more efficient than processing each sentence individually.

h. Cell 8

```
# Insert the chunks + vectors in the database
import json

with connection.cursor() as cursor:
    # Truncate the table
    cursor.execute(f"truncate table {table_name}")

    prepared_data = [(row['id'], json.dumps(row['payload']),
row['vector']) for row in data]

    # Insert the data
    cursor.executemany(
        f"""INSERT INTO {table_name} (id, payload, vector)
        VALUES (:1, :2, :3)""",
        prepared_data
    )

    connection.commit()
```

Initially, we use a `cursor` object from the established database connection to execute a command that truncates the specified table. This operation ensures that all existing rows are removed, effectively resetting the table to an empty state and preparing it for fresh data insertion.

Subsequently, the code prepares a list of tuples containing the new data. Each tuple includes an id, a JSON-encoded payload, and a vector.

The `json.dumps` function is used to convert the payload into a JSON string format, ensuring that complex data structures are properly serialized for database storage.

We then utilize the `cursor.executemany` method to insert all prepared tuples into the table in a single batch operation. This method is highly efficient for handling bulk inserts, significantly reducing the number of database transactions and improving performance. Finally, the `connection.commit` method is called to commit the transaction, ensuring that all changes are saved and made permanent in the database.

Copyright © 2024, Oracle and/or its affiliates.

i. Cell 9

```
with connection.cursor() as cursor:
    # Define the query to select all rows from a table
    query = f"SELECT * FROM {table_name}"

    # Execute the query
    cursor.execute(query)

    # Fetch all rows
    rows = cursor.fetchall()

    # Print the rows
    for row in rows[:5]:
        print(row)
```

Above will examine what is stored in the table in our database and print first 5 rows. You should be able to see embeddings for the respective text chunks.

This output should look similar to below

```
(2, {'text': 'faq | Who should use Oracle Cloud Free
Tier?\n\nOracle Cloud Free Tier services are for everyone.
Whether you're a developer building and testing applications, a
startup founder creating new systems with the intention of
scaling later, an enterprise looking to test things before
moving to cloud, a student wanting to learn, or an academic
developing curriculum in the cloud, Oracle Cloud Free Tier
enables you to learn, explore, build and test for free.',
'path': 'faq'}, array('f', [0.0475485734641552, -
0.02257898636162281, 0.014575185254216194, -
0.021478625014424324, 0.012853561900556087, -
0.013652878813445568, -0.07855578511953354, 0.05885819345712662,
-0.0038798199966549873, 0.002898386213928461,
0.02300194837152958, 0.021663479506969452, -
0.014754838310182095, -0.05274977907538414,
0.029338177293539047, -0.059429824352264404,
0.02058180421590805, 0.0016598980873823166,
0.004078279715031385, -0.05929173156619072, -
0.049143239855766296, -0.03091178…
```

5. Once all the cells above run successfully, go to **File > Save Notebook** to save the notebook.

Copyright © 2024, Oracle and/or its affiliates.

Generative AI Labs: RAG using Oracle Database 23ai - Retrieval

Lab 3-01 Practices

Get Started

Overview

This lab guides you through the steps to integrate the vector database (Oracle Database 23ai in our case) and retrieve a list of text chunks that are close to the "question" in vector space. Then, we will use the most relevant text chunks to create an LLM prompt and ask the Oracle Generative AI Service to create a nicely worded response for us.

This is a classical Retrieval-Augmented Generation (RAG) approach. The Retrieval-Augmented Generation architecture combines retrieval-based and generation-based methods to enhance natural language processing tasks. It consists of a retriever, which searches a knowledge base for relevant documents, and a generator, which uses these documents to produce informed responses. This dual approach improves accuracy and detail compared to models relying solely on pre-trained knowledge.

Retriever: The retriever employs find the most pertinent documents by embedding both queries and documents into the same vector space. This ensures that the top N relevant documents are selected to provide additional context for the generator. By doing so, the retriever enhances the quality of the generated text, particularly for queries needing specific or up-to-date information.

Generator: The generator, based on models like LLaMA 2, integrates the retrieved documents into its response generation. It may concatenate the query with the retrieved texts or use attention mechanisms to focus on relevant parts, producing coherent and contextually rich responses.

This combination allows RAG to handle diverse tasks, making it a versatile tool in natural language processing.

In this lab, you will:

- Create a Jupyter notebook to

    a) Retrieve records from a vector database based on vector distance from a "question".

    b) Create a large language model prompt using the question, the top chunks from the knowledge base, and custom instructions.

    c) Use the prompt to call the Oracle Generative AI Service and get a response from an LLM model.

Copyright © 2024, Oracle and/or its affiliates.

Prerequisites

- You have completed all the practices from **Lab0-01** and **Lab3-01**.

Assumptions

- You are familiar with Python coding language and SQL queries.
- You are on the JupyterLab screen in your browser and have already updated the **config.txt** as directed in the step 4 of the **Start a JupyterLab Server** practice in the **Lab0-01**.
- You can see the **LoadProperties.py, config.txt, faq.txt (inside txt-docs folder)** and **oci-ai-foundations.pdf (inside pdf-docs folder)** with few other files in the File Browser section on the left-hand side.

Copyright © 2024, Oracle and/or its affiliates.

Create a Jupyter Notebook

JupyterLab is the latest web-based interactive development environment for notebooks, code, and data. Its flexible interface allows users to configure and arrange workflows in data science and machine learning.

Notebook provides a simple, linear notebook interface where you can write and execute code cells, see output inline, and use markdown for text.

Tasks

1. In the Console ribbon at the top of the JupyterLab Server screen in your browser, click on the **File** > **New** > **Notebook**.
2. This will open a **Select Kernel** dialog box. Leave the default kernel (**Python 3**) as it is and click on **Select**.
3. You will see an **Untitled.ipynb** notebook created. Right click on it and select **Rename**. Rename the notebook to **Lab4.ipynb**.
4. Once renamed, in the cell start copy these code snippets one by one and execute them by pressing **shift + enter**. Once executed you will see that the cell has been assigned a number. You may see an output below the cell depending on the code written.

   a. Cell 1

```
table_name = 'faqs'
topK = 3

sql = f"""select payload, vector_distance(vector, :vector,
COSINE) as score
from {table_name}
order by score
fetch approx first {topK} rows only"""
```

In the above SQL query, `topK` represents the number of top results to retrieve. The query selects the payload column along with the cosine distance between the vector column in the specified table `{table_name}` and a provided vector parameter `:vector`, aliasing the distance calculation as `score`.

By ordering the results by the calculated `score` and using `fetch approx first {topK} rows only`, the query efficiently retrieves only the top `topK` results based on their cosine similarity to the provided vector.

Copyright © 2024, Oracle and/or its affiliates.

b. Cell 2

```
# Define the query
question = 'What is Always Free?'
```

We will then define the user query.

c. Cell 3

```
# Connect to the Oracle Database 23ai
un = "vector"
pw = "vector"
cs = "localhost/FREEPDB1"

import oracledb

connection = oracledb.connect(user=un, password=pw, dsn=cs)
```

`oracledb.connect()` function establishes a connection to an Oracle database using the provided credentials and connection details. The function takes username, password and dsn. The DSN (data source name) specifies the host, port and database service name to connect to. You can refer the bash script **init-genailabs.sh** which you earlier downloaded in step 1 of the tasks of Download the Bash Script File practice in **Lab0-01**.

d. Cell 4

```
from sentence_transformers import SentenceTransformer
encoder = SentenceTransformer('all-MiniLM-L12-v2')
```

We need an encoder to handle the vectorization for us. `all-MiniLM-L12-v2` is a specific pre-trained model that is designed to be an encoder. It is based on the MiniLM (Mini Language Model) architecture, which is a lightweight version of transformer models like BERT.

Note:

- Ignore the warning saying IProgress not found., among others.

e. Cell 5

```
# Retrieval Code
import array
import json

with connection.cursor() as cursor:
  embedding = list(encoder.encode(question))
```

Copyright © 2024, Oracle and/or its affiliates.

```
vector = array.array("f", embedding)

results  = []

for (info, score, ) in cursor.execute(sql, vector=vector):
    text_content = info.read()
    results.append((score, json.loads(text_content)))
```

Next, we write the retrieval code. We employ the same encoder as in previous text chunks, generating a vector representation of the question.

The SQL query is executed with the provided vector parameter, fetching relevant information from the database. For each result, the code retrieves the text content, stored in JSON format, and appends it to a list along with the calculated similarity score. This process iterates through all fetched results, accumulating them in the results list.

f. Cell 6

```
# Check results
import pprint
pprint.pp(results)
```

If we print the results using above, we obtain something like the following.

```
[(0.342059188035412, {'text': 'faq | What are Always Free
services?\n' '\n' 'Always Free services are part of Oracle Cloud
Free Tier. Always ' 'Free services are available for an
unlimited time. Some ' 'limitations apply. As new Always Free
services become available, ' 'you will automatically be able to
use those as well.\n' '\n' 'The following services are available
as Always Free:\n' 'Email Delivery', 'path': 'faq'})]
```

As requested, we have the "score" of each hit, which is essentially the distance in vector space between the question and the text chunk, as well as the metadata JSON embedded in each chunk.

g. Cell 7

```
from transformers import LlamaTokenizerFast
import sys

tokenizer = LlamaTokenizerFast.from_pretrained("hf-internal-
testing/llama-tokenizer")
```

Copyright © 2024, Oracle and/or its affiliates.

```
tokenizer.model_max_length = sys.maxsize

def truncate_string(string, max_tokens):
    # Tokenize the text and count the tokens
    tokens = tokenizer.encode(string, add_special_tokens=True)
    # Truncate the tokens to a maximum length
    truncated_tokens = tokens[:max_tokens]
    # transform the tokens back to text
    truncated_text = tokenizer.decode(truncated_tokens)
    return truncated_text
```

Before sending anything to the LLM, we must ensure that our prompt does not exceed the maximum context length of the model. We are planning to use LLaMA 2, so the context is limited to 4,096 tokens. Note that the context is used for both the input tokens (the prompt) and the response.

Above code leverages the Hugging Face Transformers library to tokenize text using the `LlamaTokenizerFast` model. The tokenizer is initialized from the pre-trained `hf-internal-testing/llama-tokenizer` model, and its `model_max_length` attribute is set to `sys.maxsize` to handle extremely large inputs without length constraints.

The `truncate_string` function takes a string and a maximum token count as inputs. It tokenizes the input string, truncates the tokenized sequence to the specified maximum length, and then decodes the truncated tokens back into a string. This function effectively shortens the text to a specified token limit while preserving its readable format, useful for tasks requiring length constraints on input text.

Note: Ignore the legacy warning.

h. Cell 8

```
import os

def loadFAQs(directory_path):
   faqs = {}

   for filename in os.listdir(directory_path):
      if filename.endswith(".txt"):  # assuming FAQs are in .txt
files
         file_path = os.path.join(directory_path, filename)
```

Copyright © 2024, Oracle and/or its affiliates.

```
        with open(file_path) as f:
           raw_faq = f.read()

        filename_without_ext = os.path.splitext(filename)[0]  #
remove .txt extension
        faqs[filename_without_ext] = [text.strip() for text in
raw_faq.split('=====')]

   return faqs

faqs = loadFAQs('./txt-docs')

docs = [{'text': filename + ' | ' + section, 'path': filename}
for filename, sections in faqs.items() for section in sections]
```

We will now read, split and store the data. We had already seen above code in the steps 4-a, 4-b and 4-c of the Lab3-01 Tasks. Please refer to the same for details.

i. Cell 9

```
# Transform docs into a string array using the "paylod" key
docs_as_one_string = "\n=========\n".join([doc["text"] for doc
in docs])
docs_truncated = truncate_string(docs_as_one_string, 1000)
```

We will truncate our chunks to 1000 tokens, to leave plenty of space for the rest of the prompt and the answer.

j. Cell 10

```
# Create the LLM Prompt
prompt = f"""\
    <s>[INST] <<SYS>>
    You are a helpful assistant named Oracle chatbot.
    USE ONLY the sources below and ABSOLUTELY IGNORE any
previous knowledge.
    Use Markdown if appropriate.
    Assume the customer is highly technical.
    <</SYS>> [/INST]

    [INST]
    Respond to PRECISELY to this question: "{question}.",  USING
ONLY the following information and IGNORING ANY PREVIOUS
KNOWLEDGE.
    Include code snippets and commands where necessary.
```

Copyright © 2024, Oracle and/or its affiliates.

```
    NEVER mention the sources, always respond as if you have
that knowledge yourself. Do NOT provide warnings or disclaimers.
    =====
    Sources: {docs_truncated}
    =====
    Answer (Three paragraphs, maximum 50 words each, 90%
spartan):
    [/INST]
    """
```

The prompt will include the retrieved top chunks, the question posed by the user, and the custom instructions.

k. Cell 11

```
import oci
from LoadProperties import LoadProperties

# Setup basic variables
properties = LoadProperties()

# Use Instance Principals for Authentication
signer =
oci.auth.signers.InstancePrincipalsSecurityTokenSigner()

generative_ai_inference_client =
oci.generative_ai_inference.GenerativeAiInferenceClient(config={
}, signer=signer, service_endpoint=properties.getEndpoint(),
retry_strategy=oci.retry.NoneRetryStrategy(), timeout=(10,240))
chat_detail = oci.generative_ai_inference.models.ChatDetails()
chat_request =
oci.generative_ai_inference.models.CohereChatRequest()
chat_request.message = prompt
chat_request.max_tokens = 1000
chat_request.temperature = 0.0
chat_request.frequency_penalty = 0
chat_request.top_p = 0.75
chat_request.top_k = 0

chat_detail.serving_mode =
oci.generative_ai_inference.models.OnDemandServingMode(model_id=
properties.getModelName())
chat_detail.chat_request = chat_request
chat_detail.compartment_id = properties.getCompartment()
chat_response = generative_ai_inference_client.chat(chat_detail)
```

Copyright © 2024, Oracle and/or its affiliates.

We will now call the OCI Generative AI Chat model and store the chat model's response in `chat_response`.

Please refer to the Lab1-01 practices for the details.

I. Cell 12

```
pprint.pp(
    chat_response.data.chat_response.chat_history[1].message
)
```

The response is extracted and cleaned of any leading or trailing whitespace before being printed in a readable format.

You should see a response similar to below

```
('Always Free is a program offered by Oracle Cloud. It provides
users with '

 'access to a range of services that are completely free to use
indefinitely. '

 'These include compute, storage, and networking resources. \n'

 '\n'

 'Always Free is ideal for developers who want to experiment,
build, and test '

 'applications in the cloud without incurring any costs. The
program is '

 'available to anyone, and you can sign up for it on the Oracle
Cloud Free '

 'Tier webpage.')
```

5. Once all the cells above run successfully, go to **File > Save Notebook** to save the notebook.

Copyright © 2024, Oracle and/or its affiliates.

Generative AI Labs: Build a Chatbot using Oracle Database 23ai and LangChain

Lab 5-01 Practices

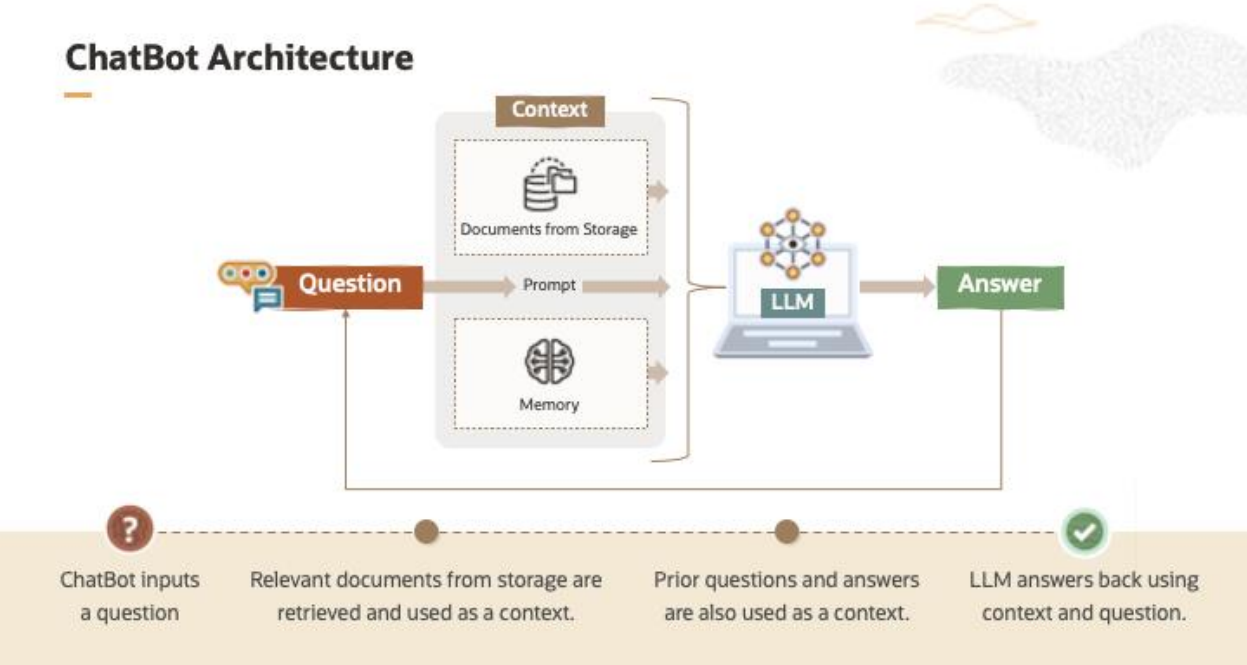
Get Started

Overview

Labs 5 guides you to build a chatbot using Oracle 23 ai vector database and LangChain framework. We will build a chatbot to answer questions on a provided PDF document.

This lab guide (Lab 5) is split into two parts. The first practice will walk you through code required to load and parse a PDF file named **oci-ai-foundations.pdf** from the **pdf-docs** local folder, split them into chunks, create embeddings and store them into the **Oracle 23 ai** vector database.

In the second practice, we will learn how to answer a user query by retrieving relevant embedded chunks from the Oracle 23 ai vector store. We will also consider a list of prior questions and answers, i.e. chat history for answering a query.



In this lab, you will:

- Create and run a Jupyter notebook to split a **oci-ai-foundations.pdf** into chunks and store them as embedded vectors using OCI Generative AI service embedding model in a vector database (Oracle Database 23ai).
- Create and run a Jupyter notebook to answer user query using relevant document chunks from Oracle 23 ai vector store and chat history.

Copyright © 2024, Oracle and/or its affiliates.

Prerequisites

- You have completed all the practices from **Lab0-01**.

Assumptions

- You are familiar with Python coding language and LangChain.
- You are on the JupyterLab screen in your browser and have already updated the **config.txt** as directed in the step 4 of the **Start a JupyterLab Server** practice in the **Lab0-01**.
- You can see the **LoadProperties.py, config.txt, faq.txt (inside txt-docs folder)** and **oci-ai-foundations.pdf (inside pdf-docs folder)** with few other files in the File Browser section on the left-hand side.

Copyright © 2024, Oracle and/or its affiliates.

Ingest PDF document to Oracle 23 ai vector store

JupyterLab is the latest web-based interactive development environment for notebooks, code, and data. Its flexible interface allows users to configure and arrange workflows in data science and machine learning.

Notebook provides a simple, linear notebook interface where you can write and execute code cells, see output inline, and use markdown for text.

Tasks

1. In the Console ribbon at the top of the JupyterLab Server screen in your browser, click on the **File** > **New** > **Notebook**.
2. This will open a **Select Kernel** dialog box. Leave the default kernel (**Python 3**) as it is and click on **Select**.
3. You will see an **Untitled.ipynb** notebook created. Right click on it and select **Rename**. Rename the notebook to **Lab5-practice1.ipynb**.
4. Once renamed, in the cell start copy these code snippets one by one and execute them by pressing **shift + enter**. Once executed you will see that the cell has been assigned a number. You may see an output below the cell depending on the code written.

   a. Cell 1

```
from pypdf import PdfReader

import oracledb

import oci

from langchain.text_splitter import CharacterTextSplitter

from langchain_community.vectorstores.oraclevs import OracleVS

from langchain_community.embeddings import OCIGenAIEmbeddings

from langchain_community.vectorstores.utils import
DistanceStrategy

from langchain_core.documents import BaseDocumentTransformer,
Document

from LoadProperties import LoadProperties

print("Successfully imported libraries and modules")
```

   In the above block of code, we import required libraries including pypdf, oracledb, oci and langchain.

Copyright © 2024, Oracle and/or its affiliates.

b. Cell 2

```
# Setup basic variables
properties = LoadProperties()

# Use Instance Principals for Authentication
signer =
oci.auth.signers.InstancePrincipalsSecurityTokenSigner()

#Declare username and password and dsn (data connection string)
un = "vector"
pw = "vector"
cs = "localhost/FREEPDB1"

# Connect to the database
try:
    conn23c = oracledb.connect(user=un, password=pw, dsn=cs)
    print("Connection successful!")
except Exception as e:
    print("Connection failed!")
```

In the above block of code, we use LoadProperties class to read **config.txt** and load model name, service endpoint, embedding model name and compartment ocid. Next, we declare username , password and database connection string. Finally, we create a database connection using username, password and connection string.

c. Cell 3

```
# RAG Step 1 - Load the document and create pdf reader object

pdf = PdfReader('./pdf-docs/oci-ai-foundations.pdf')

# RAG Step 2 - Transform the document to text

text=""
for page in pdf.pages:
    text += page.extract_text()
```

Copyright © 2024, Oracle and/or its affiliates.

```
print("You have transformed the PDF document to text format")


# RAG Step 3 - Chunk the text document into smaller chunks

text_splitter =
CharacterTextSplitter(separator=".",chunk_size=2000,chunk_overla
p=100)

chunks = text_splitter.split_text(text)
```

In the above block of code, we extract text of all the pages of a PDF which is **oci-ai-foundations.pdf** in this case and split it into chunks of 2000 characters with the overlap of 100 characters. This ensures that chunks do not loose continuity of the context.

d. Cell 4

```
# Function to format and add metadata to Oracle 23ai Vector
Store


def chunks_to_docs_wrapper(row: dict) -> Document:

    """

    Converts text into a Document object suitable for ingestion
into Oracle Vector Store.

    - row (dict): A dictionary representing a row of data with
keys for 'id', 'link', and 'text'.

    """

    metadata = {'id': row['id'], 'link': row['link']}

    return Document(page_content=row['text'], metadata=metadata)


# RAG Step 4 - Create metadata wrapper to store additional
information in the vector store

"""

Converts a row from a DataFrame into a Document object suitable
for ingestion into Oracle Vector Store.

- row (dict): A dictionary representing a row of data with keys
for 'id', 'link', and 'text'.

"""
```

Copyright © 2024, Oracle and/or its affiliates.

```
docs = [chunks_to_docs_wrapper({'id': str(page_num), 'link':
f'Page {page_num}', 'text': text}) for page_num, text in
enumerate(chunks)]
```

In the above block of code, we declare chunks_to_docs_wrapper function to add metadata to the chunks and convert the chunk text and meta data to Document object. We iterate through all the chunks, extract page number and text from each chunk and set these in the metadata.

e. Cell 5

```
# RAG Step 5 - Using an embedding model, embed the chunks as
vectors into Oracle Database 23ai.


embed_model = OCIGenAIEmbeddings(

    model_id=properties.getEmbeddingModelName(),

    service_endpoint=properties.getEndpoint(),

    compartment_id=properties.getCompartment(),

    auth_type="INSTANCE_PRINCIPAL",

)


# RAG Step 6 - Configure the vector store with the model, table
name, and using the indicated distance strategy for the
similarity search and vectorize the chunks


knowledge_base = OracleVS.from_documents(docs, embed_model,
client=conn23c, table_name="DEMO_TABLE",
distance_strategy=DistanceStrategy.DOT_PRODUCT )

print("Chunks are stored in the DEMO_TABLE")
```

In the above block of code, we use OCIGenAIEmbeddings to create an embed_model object. We pass in model name, service endpoint and compartment ocid. We have set policies for our VM to access OCI Generative AI service, hence we use auth_type as INSTANCE_PRINCIPAL. Next, we embed and save all the docs in the DEMO_TABLE of the Oracle 23 ai database using from_documents method of the OracleVS class. We will use DOT_PRODUCT to compute similarity between embeddings.

5. Once all the cells above run successfully, go to **File > Save Notebook** to save the notebook.

Copyright © 2024, Oracle and/or its affiliates.

Implement chat using RAG and chat history

JupyterLab is the latest web-based interactive development environment for notebooks, code, and data. Its flexible interface allows users to configure and arrange workflows in data science and machine learning.

Notebook provides a simple, linear notebook interface where you can write and execute code cells, see output inline, and use markdown for text.

Tasks

1. In the Console ribbon at the top of the JupyterLab Server screen in your browser, click on the **File** > **New** > **Notebook**.
2. This will open a **Select Kernel** dialog box. Leave the default kernel (**Python 3**) as it is and click on **Select**.
3. You will see an **Untitled.ipynb** notebook created. Right click on it and select **Rename**. Rename the notebook to **Lab5-practice2.ipynb**.
4. Once renamed, in the cell start copy these code snippets one by one and execute them by pressing **shift + enter**. Once executed you will see that the cell has been assigned a number.

   a. Cell 1

```
%%writefile chatbot-streamlit.py

import oracledb

import oci

from langchain.memory import ConversationBufferMemory

from langchain.chains import ConversationalRetrievalChain

from langchain_community.vectorstores.utils import
DistanceStrategy

from langchain_community.vectorstores.oraclevs import OracleVS

from langchain_community.chat_models.oci_generative_ai import
ChatOCIGenAI

from langchain_community.embeddings import OCIGenAIEmbeddings

from LoadProperties import LoadProperties


print("Successfully imported libraries and modules")
```

Copyright © 2024, Oracle and/or its affiliates.

```
# Setup basic variables
properties = LoadProperties()

#Declare username and password and dsn (data connection string)
un = "vector"
pw = "vector"
cs = "localhost/FREEPDB1"

# Connect to the database
try:
    conn23c = oracledb.connect(user=un, password=pw, dsn=cs)
    print("Connection successful!")
except Exception as e:
    print("Connection failed!")

#In this demo we will explore using ConversationalRetrievalChain
chain to retrieve relevant documents and send these as a context
in a query.
# We will use Chroma vectorstore.

#Step 1 - here we create a function that creates a chain

def create_chain():
    embed_model = OCIGenAIEmbeddings(
        model_id=properties.getEmbeddingModelName(),
        service_endpoint=properties.getEndpoint(),
        compartment_id=properties.getCompartment(),
        auth_type="INSTANCE_PRINCIPAL",
    )
```

Copyright © 2024, Oracle and/or its affiliates.

```
    vs = OracleVS(embedding_function=embed_model,
client=conn23c, table_name="DEMO_TABLE",


distance_strategy=DistanceStrategy.DOT_PRODUCT)



    retv = vs.as_retriever(search_type="similarity",
search_kwargs={'k': 3})


    llm = ChatOCIGenAI(

        model_id=properties.getModelName(),

        service_endpoint=properties.getEndpoint(),

        compartment_id=properties.getCompartment(),

        auth_type="INSTANCE_PRINCIPAL",

        model_kwargs={"max_tokens":200}

        )

    memory = ConversationBufferMemory(llm=llm,
memory_key="chat_history", return_messages=True,
output_key='answer')

    qa = ConversationalRetrievalChain.from_llm(llm,
retriever=retv, memory=memory, return_source_documents=True)

    return qa




#Step 2 - here we declare a chat function

def chat(llm_chain, user_input):

    # generate a prediction for a prompt

    bot_json = llm_chain.invoke(user_input)

    return {"bot_response": bot_json}


#Step 3 - here we setup Streamlit text input and pass input text
to chat function.

# chat function returns the response and we print it.
```

Copyright © 2024, Oracle and/or its affiliates.

```
if __name__ == "__main__":
    import streamlit as st

    st.subheader("Chatbot that answers your study questions")
    col1 , col2 = st.columns([4,1])

    def initialize_session_state():
        if "llm_chain" not in st.session_state:
            st.session_state["llm_chain"] = create_chain()
            llm_chain = st.session_state["llm_chain"]
        else:
            llm_chain = st.session_state["llm_chain"]
        return llm_chain

    user_input = st.chat_input()
    with col1:
        col1.subheader("------Ask me a question about science
chapters------")
        #col2.subheader("References")
        if "messages" not in st.session_state:
            st.session_state.messages = []
        if user_input:
            llm_chain = initialize_session_state()
            bot_response = chat(llm_chain, user_input)
            print("bot_response->\n", bot_response)
            st.session_state.messages.append({"role" :
"chatbot", "content" : bot_response})
            #st.write("OU Assistant Response: ", bot_response)
            for message in st.session_state.messages:
                st.chat_message("user")
                st.write("Question: ",
message['content']['bot_response']['question'])
```

Copyright © 2024, Oracle and/or its affiliates.

```
                st.chat_message("assistant")

                st.write("Answer: ",
message['content']['bot_response']['answer'])

            #with col2:

                st.chat_message("assistant")

                for doc in
message['content']['bot_response']['source_documents']:

                    st.write("Chunk Id Is: ", doc.metadata['id']
+ "  \n"+ "-page->"+str(doc.metadata['link']))
```

You will see that file browser on the left shows **chatbot-streamlit.py** is created after executing Cell 1.

Here is the walkthrough of the code of Cell 1:

(**Do not copy these code blocks till Cell 2 to the notebook, you have already copied and executed these in the earlier step already**)

```
%%writefile chatbot-streamlit.py

import oracledb

import oci

from langchain.memory import ConversationBufferMemory

from langchain.chains import ConversationalRetrievalChain

from langchain_community.vectorstores.utils import
DistanceStrategy

from langchain_community.vectorstores.oraclevs import OracleVS

from langchain_community.chat_models.oci_generative_ai import
ChatOCIGenAI

from langchain_community.embeddings import OCIGenAIEmbeddings

from LoadProperties import LoadProperties


print("Successfully imported libraries and modules")


# Setup basic variables

properties = LoadProperties()
```

Copyright © 2024, Oracle and/or its affiliates.

```
#Declare username and password and dsn (data connection string)
un = "vector"
pw = "vector"
cs = "localhost/FREEPDB1"

# Connect to the database
try:
    conn23c = oracledb.connect(user=un, password=pw, dsn=cs)
    print("Connection successful!")
except Exception as e:
    print("Connection failed!")
```

In the above block of code, we write the contents of a cell as a python script using %%writefile magic command. We begin with import of libraries, next we declare username, password and connection string which we use to create a database connection. Finally, we create a database connection.

```
#Step 1 - here we create a function that creates a chain

def create_chain():
    embed_model = OCIGenAIEmbeddings(
        model_id=properties.getEmbeddingModelName(),
        service_endpoint=properties.getEndpoint(),
        compartment_id=properties.getCompartment(),
        auth_type="INSTANCE_PRINCIPAL",
    )

    vs = OracleVS(embedding_function=embed_model,
client=conn23c, table_name="DEMO_TABLE",

distance_strategy=DistanceStrategy.DOT_PRODUCT)
```

Copyright © 2024, Oracle and/or its affiliates.

```
    retv = vs.as_retriever(search_type="similarity",
search_kwargs={'k': 3})


    llm = ChatOCIGenAI(

        model_id=properties.getModelName(),

        service_endpoint=properties.getEndpoint(),

        compartment_id=properties.getCompartment(),

        auth_type="INSTANCE_PRINCIPAL",

        model_kwargs={"max_tokens":200}

        )

    memory = ConversationBufferMemory(llm=llm,
memory_key="chat_history", return_messages=True,
output_key='answer')

    qa = ConversationalRetrievalChain.from_llm(llm,
retriever=retv, memory=memory, return_source_documents=True)

    return qa
```

In the above block of code, we declare a function to create a chain of the type ConversationalRetrievalChain.

This chain takes llm, retriever and memory objects as inputs. We also pass in parameter return_source_documents=True.This ensures that llm will return the relevant documents to the user query.

Memory object is created using ConversationBufferMemory class. Here we use ConversationBufferMemory class to store a chat history, i.e. a list of questions we ask and responses we get from the llm. We set return_messages as true , so that memory contents are returned as a list of messages. output_key is set as ‘answer’ , this ensures that only ‘answer’ from the memory is sent out from the memory and not the question.

To create a retriever object, vector store object is created using OracleVS class. Embedding model, connection object and name of the table where embeddings will be stored is passed to it.

```
#Step 2 - here we declare a chat function

def chat(llm_chain, user_input):

    # generate a prediction for a prompt

    bot_json = llm_chain.invoke(user_input)

    return {"bot_response": bot_json}
```

Copyright © 2024, Oracle and/or its affiliates.

In the above block of code, we create a chat function which takes in chain and user input as a parameter. User input is passed to the chain and json response is returned.

```
#Step 3 - here we setup Streamlit text input and pass input text
to chat function.

# chat function returns the response and we print it.


if __name__ == "__main__":

    import streamlit as st


    st.subheader("OU Chatbot")

    col1 , col2 = st.columns([4,1])


    def initialize_session_state():

        if "llm_chain" not in st.session_state:

            st.session_state["llm_chain"] = create_chain()

            llm_chain = st.session_state["llm_chain"]

        else:

            llm_chain = st.session_state["llm_chain"]

        return llm_chain


    user_input = st.chat_input()

    with col1:

        col1.subheader("----Ask me about OCI AI Foundations
course----")


        if "messages" not in st.session_state:

            st.session_state.messages = []

        if user_input:

            llm_chain = initialize_session_state()

            bot_response = chat(llm_chain, user_input)
```

Copyright © 2024, Oracle and/or its affiliates.

```
            print("bot_response->\n", bot_response)

            st.session_state.messages.append({"role" :
"chatbot", "content" : bot_response})

            #st.write("OU Assistant Response: ", bot_response)

            for message in st.session_state.messages:

                st.chat_message("user")

                st.write("Question: ",
message['content']['bot_response']['question'])

                st.chat_message("assistant")

                st.write("Answer: ",
message['content']['bot_response']['answer'])

                st.chat_message("assistant")

                for doc in
message['content']['bot_response']['source_documents']:

                    st.write("Chunk Id Is: ", doc.metadata['id']
+ "  \n"+ "-page->"+str(doc.metadata['link']))
```

In the above block of code, we use streamlit to layout UI , next we create a function initialize_session_state. In this function we check if a chain exists in a streamlit session, if the chain does not exist in the session, chain is create using create_chain function and put in the session.

If the chain exists, the same is returned.

Next user input is captured with st.chat_input() function. We append the bot_response to list of messages.

Finally, we parse and display the messages in the form of Q&A format along with the returned source documents.

b. Cell 2

```
!streamlit run chatbot-streamlit.py
```

Note: Execute Cell 2 by pressing **shift + enter.**

Above command runs the chatbot-streamlit.py file created earlier through Cell 1 using streamlit and display the URL for chatbot UI. You should see an output similar to below:

Collecting usage statistics. To deactivate, set browser.gatherUsageStats to false.

Copyright © 2024, Oracle and/or its affiliates.

You can now view your Streamlit app in your browser.

Local URL: **http://localhost:8501**

Network URL: **http://10.0.0.223:8501**

External URL: **http://140.184.250.149:8501/**

Successfully imported libraries and modules

Connection successful!

5. Once all the cells above run successfully, go to **File > Save Notebook** to save the notebook.

6. You can now click on the displayed external URL in the Cell 2 output

   For example: Click the

   External URL: **http://140.184.250.149:8501/**

   to open the Chatbot UI in a browser on your computer.

   Alternatively, you can open a browser on your computer and type the following:

   http://*<public-ip-address>*:8501

   (Replace the *<public-ip-address>* with **GEN-AI-LABS** instance's public ip with what you copied in the Step 5 of Create a Compute Instance practice of Lab0-01)

   Note:

   - You should now see a Chatbot UI where you can ask questions and get responses with references. Please see a sample screenshot below:

Copyright © 2024, Oracle and/or its affiliates.

OU Chatbot

----Ask me about OCI AI Foundations course----

Question: How many modules are in OCI AI Foundations course?

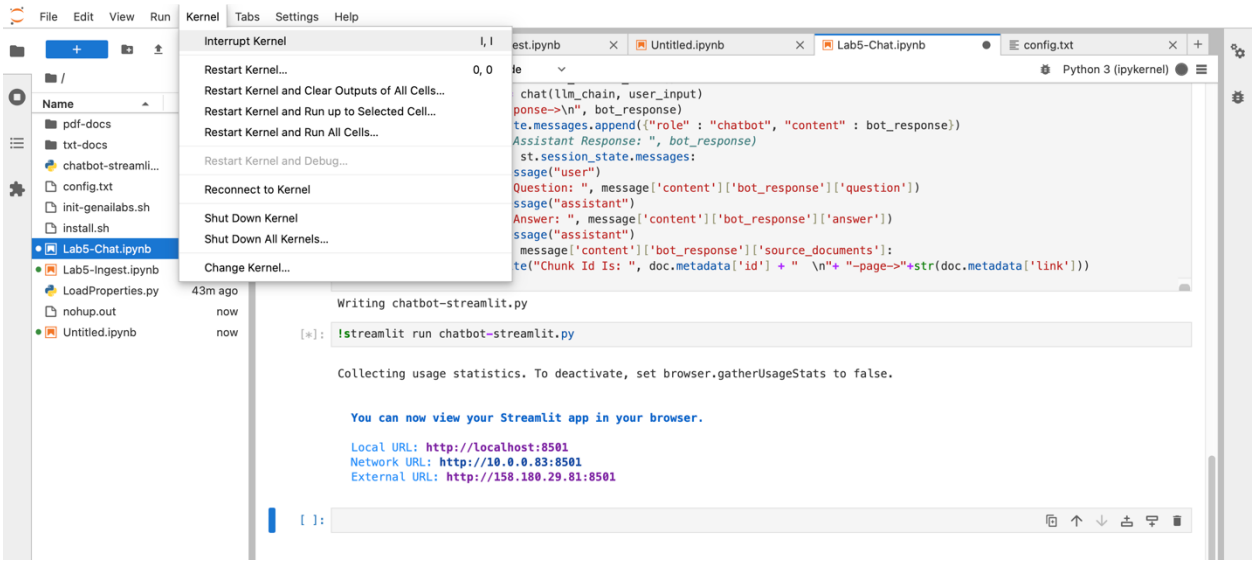
Answer: The OCI AI Foundations course is divided into 6 modules. They cover a range of topics from foundational AI concepts to OCI AI services.

Chunk Id Is: 0
-page->Page 0

Chunk Id Is: 1
-page->Page 1

Once you are done with using the Chatbot, click **Kernel -> Interrupt Kernel**

Copyright © 2024, Oracle and/or its affiliates.



Copyright © 2024, Oracle and/or its affiliates.

Purge the Compute Instance

In this practice, you will purge the compute instance created earlier to delete the associated resources and files, assuming you have completed all the labs and their individual practices (from **Lab0-01** to **Lab5-01**).

Tasks

1. From the navigation menu, under **Compute**, click on **Instances**.
2. You are now on **Instance details** page.
3. Click the **Terminate** button. Terminate Instance dialog box will open.
4. Select **Permanently delete the attached boot volume** and click **Terminate instance**.

Copyright © 2024, Oracle and/or its affiliates.

Purge the Virtual Cloud Network

In this practice, you will purge the virtual cloud network (VCN) to delete the associated resources such as subnet, assuming you have completed all the labs and their individual practices (from **Lab0-01** to **Lab5-01**).

Tasks

1. From the navigation menu, under **Networking**, click on **Virtual Cloud Networks**.
2. You are now on **Instance details** page. From the left navigation panel, change the list scope to each student's assigned compartment.
3. In the list of VCNs, click the three dots on the right of the VCN name ('GENAILABS-VCN') to open the **Actions** menu. Select **Delete**.
4. Make sure that the **Search compartments for resources associated with this VCN** checkbox is checked.
5. In the white box that starts with **Select which compartments to search for associated resources**, click the **Specific compartments** radio button and select your assigned compartment from the drop-down menu.
6. Click **Scan**.
7. Once the scan has completed, click **Delete All**.

Note:

- This should delete the VCN - 'GENAILABS-VCN' and SUBNET – 'GENAILABS-SNET' which you can cross check.
- This concludes our OCI Generative AI Labs from **Lab0-01** to **Lab5-01**.

Copyright © 2024, Oracle and/or its affiliates.

Copyright © 2024, Oracle and/or its affiliates.